

GUIA DEFINITIVO DE ESTUDO: PROGRAMAÇÃO EM PYTHON

Material de Apoio Técnico e Revisão Teórica para Provas

Este material foi estruturado para servir de consulta rápida e aprofundada dos tópicos fundamentais e avançados da linguagem Python. Domine os conceitos de lógica, estruturas de dados e tratamento de exceções através dos exemplos práticos documentados.

1. O MODELO IPO (INPUT - PROCESS - OUTPUT)

O modelo **IPO** fundamenta a arquitetura lógica de quase todo algoritmo ou sistema computacional, dividindo o fluxo de execução em três etapas bem definidas:

- **Input (Entrada):** Captação dos dados externos necessários para o funcionamento (ex: teclados, sensores, APIs, arquivos).
- **Process (Processamento):** A manipulação computacional desses dados (ex: cálculos matemáticos, estruturas lógicas de decisão, transformações).
- **Output (Saída):** O retorno gerado e enviado ao usuário ou a outro sistema (ex: textos no console, geração de relatórios, persistência em bancos de dados).

```
# --- EXEMPLO PRÁTICO DO MODELO IPO ---  
  
# 1. Input: Entrada de dados (por padrão, retorna sempre como String)  
raio_str = input("Digite o raio do círculo: ")  
raio = float(raio_str) # Conversão explícita (coerção de tipo)  
  
# 2. Process: Processamento e cálculo de dados  
pi = 3.14159  
area = pi * (raio ** 2)  
  
# 3. Output: Saída formatada dos dados convertidos  
print(f"A área do círculo é: {area:.2f}")
```

2. TIPOS DE DADOS, VARIÁVEIS E OPERADORES

Python é caracterizado por possuir **tipagem dinâmica** (o interpretador infere o tipo da variável automaticamente no momento da atribuição) e **fortemente tipada** (operações entre tipos incompatíveis geram erros em tempo de execução, exigindo conversão explícita).

Principais Tipos Primitivos

Tipo de Dado	Representação em Python	Exemplo Clínico / Sintaxe
Inteiro	<code>int</code>	<code>idade = 25</code> , <code>temperatura = -3</code>
Ponto Flutuante (Decimal)	<code>float</code>	<code>preco = 99.90</code> , <code>pi = 3.14159</code>
Cadeia de Caracteres (Texto)	<code>str</code>	<code>nome = "Python"</code> , <code>letra = 'A'</code>
Booleano (Lógico)	<code>bool</code>	<code>ativo = True</code> , <code>valido = False</code>

Operadores Aritméticos, de Comparação e Lógicos

```
# Atribuições iniciais
x = 10
y = 3

# Operadores Aritméticos fundamentais
soma = x + y           # 13
subtracao = x - y      # 7
multiplicacao = x * y  # 30
divisao = x / y        # 3.3333... (sempre retorna float)
divisao_inteira = x // y # 3 (desconsidera a fração decimal)
resto_divisao = x % y   # 1 (operador módulo, útil para testar pares/ímpares)
exponenciacao = x ** y  # 1000 (10 elevado ao cubo)

# Operadores de Comparação (Retornam estritamente True ou False)
e_igual = (x == y)      # False
e_diferente = (x != y)  # True
maior_que = (x > y)     # True

# Operadores Lógicos (Avaliação em curto-circuito)
# and: retorna True apenas se ambos forem verdadeiros
# or: retorna True se pelo menos um for verdadeiro
# not: inverte o valor lógico atual
condicao = (x > 5) and (y < 5) # True, pois ambas as sentenças são verdadeiras
```

3. ESTRUTURAS DE DECISÃO (IF / ELIF / ELSE / MATCH)

As estruturas condicionais controlam o desvio do fluxo de execução do programa baseado na avaliação de expressões booleanas.

Condicionais Tradicionais: if, elif, else

```
if idade < 16:
    print("Não eleitor.")
elif idade >= 18 and idade < 70:
    print("Eleitor obrigatório.")
else:
    print("Eleitor opcional.")
```

Estrutura de Correspondência de Padrões: match-case (Python 3.10+)

Equivalente ao **switch-case** de outras linguagens. O caractere curinga `_` é utilizado para capturar qualquer caso que não tenha correspondido às opções anteriores (caso padrão/default).

```
status_codigo = 404

match status_codigo:
    case 200:
        print("Requisição bem-sucedida.")
    case 404:
        print("Recurso não encontrado.")
    case 500:
        print("Erro interno do servidor.")
    case _:
        print("Código de status desconhecido.")
```

4. ESTRUTURAS DE REPETIÇÃO (FOR E WHILE)

Servem para automatizar tarefas repetitivas, executando blocos de códigos sob regras específicas:

- **while** : Executa o bloco *enquanto* uma condição for verdadeira. Ideal quando o número total de iterações não é conhecido previamente.
- **for** : Itera sobre coleções ou sequências definidas (como listas ou ranges de números).

```
# --- ESTRUTURA WHILE ---
contador = 1
while contador <= 3:
    print(f"Iteração número: {contador}")
    contador += 1 # Incremento essencial para evitar loops infinitos

# --- ESTRUTURA FOR COM A FUNÇÃO RANGE ---
# range(início, fim_exclusivo, passo)
for i in range(0, 6, 2):
    print(f"Número par: {i}") # Exibe 0, 2 e 4

# --- CONTROLE AVANÇADO DE LOOPS: BREAK E CONTINUE ---
for num in range(1, 10):
    if num == 5:
        continue # Interrompe a iteração atual e salta para o próximo número
    if num == 8:
        break # Aborta completamente a execução do loop inteiro
    print(f"Processando: {num}")
```

5. MANIPULAÇÃO DE STRINGS

Strings em Python são coleções de caracteres **imutáveis** (uma vez criadas, seus valores individuais não podem ser modificados diretamente). Elas suportam indexação direta e fatiamento sofisticado (slicing).

```
frase = " Aprender Python é Essencial! "
```

Métodos principais de String (Retornam uma nova String modificada)

```
print(frase.upper())      # Caixa alta completa
print(frase.lower())      # Caixa baixa completa
print(frase.strip())      # Remove espaços em branco do início e fim
print(frase.replace("Essencial", "Fácil")) # Substituição de substring
```

Fatiamento Avançado (Slicing) -> string[início:fim_exclusivo:passo]

```
termo = "Desenvolvimento"
print(termo[0:5])         # "Desen" (Índices de 0 a 4)
print(termo[::-1])        # "otnemivlovneseD" (Inversão completa de string)
```

Formatação moderna (f-strings)

```
linguagem = "Python"
print(f"Estou estudando {linguagem} para as avaliações.")
```

6. ESTRUTURAS DE DADOS COLECIONÁVEIS

Abaixo estão descritas as quatro principais coleções built-in do Python, detalhando suas restrições e casos de uso mais comuns.

Coleção	Sintaxe Base	Ordenada?	Mutável?	Duplicados?
Lista (List)	<code>[a, b, c]</code>	Sim	Sim	Permite
Tupla (Tuple)	<code>(a, b, c)</code>	Sim	Não	Permite
Conjunto (Set)	<code>{a, b, c}</code>	Não	Sim	Não Permite
Dicionário (Dict)	<code>{chave: valor}</code>	Sim (3.7+)	Sim	Chaves únicas

```
# --- LISTAS (Manipulação dinâmica) ---
tecnologias = ["SQL", "Docker"]
tecnologias.append("Python") # Adiciona elemento no fim da lista
tecnologias[0] = "PostgreSQL" # Substitui elemento pelo índice

# --- TUPLAS (Segurança de dados / Constantes de execução) ---
localizacao = (-23.5505, -46.6333)
# localizacao[0] = 0 -> Irá gerar um erro TypeError se executado

# --- SETS / CONJUNTOS (Operações matemáticas de união/interseção) ---
codigos = {10, 20, 30, 10}
print(codigos) # Saída: {10, 20, 30} (Remoção automática de duplicidade)

# --- DICIONÁRIOS (Mapeamentos Chave: Valor) ---
colaborador = {
    "nome": "Rodrigo",
    "cargo": "Analista de Dados",
    "salario": 4500.00
}
print(colaborador["nome"]) # Acesso via chave
colaborador["senioridade"] = "Pleno" # Inserção dinâmica de nova chave
```

7. COMBINANDO ESTRUTURAS DE DADOS

Aplicações do mundo real exigem aninhamento complexo de dados. É extremamente comum gerenciar listas de dicionários ou dicionários cujos valores mapeiam listas internas.

```
# Representação de um inventário real de produtos
catalogo = [
    {"id": 1, "nome": "Teclado Mecânico", "preco": 350.00, "tags": ["hardware", "gamer"]},
    {"id": 2, "nome": "Monitor 4K", "preco": 2100.00, "tags": ["video", "office"]},
    {"id": 3, "nome": "Cabo HDMI", "preco": 45.00, "tags": ["acessorio"]}
]

print("=== ANÁLISE DE PRODUTOS PREMIUM ===")
for produto in catalogo:
    if produto["preco"] > 300.00:
        print(f"Produto: {produto['nome']} | Categorias: {' | '.join(produto['tags'])}")
```

8. MODULARIZAÇÃO COM FUNÇÕES E ESCOPO

A modularização permite dividir grandes códigos em subprogramas menores e isolados. Funções são declaradas com a palavra reservada **def** e seguem o princípio do encapsulamento de escopo.

```
# Escopo Global: Variável acessível em qualquer ponto do script
fator_imposto = 0.05

def calcular_preco_final(valor_base):
    # Escopo Local: A variável 'preco_calculado' só existe aqui dentro
    preco_calculado = valor_base + (valor_base * fator_imposto)
    return preco_calculado

resultado_venda = calcular_preco_final(200.00)
print(f"Preço de venda finalizado: R$ {resultado_venda:.2f}")
```

 **Pegadinha de Prova sobre Escopo:** Se você tentar invocar a variável **preco_calculado** diretamente fora da função, o interpretador gerará um erro do tipo **NameError**, pois variáveis locais são destruídas da memória assim que o bloco da função termina sua execução.

9. BIBLIOTECAS, MÓDULOS E DEPURAÇÃO (DEBUGGING)

Módulos são arquivos contendo códigos reutilizáveis que podem ser importados para o seu script. O ecossistema padrão do Python fornece módulos robustos sem necessidade de instalações externas.

```
import math
import random

# Utilizando funções da biblioteca matemática
print(math.sqrt(25))          # Raiz quadrada -> 5.0
print(math.ceil(4.2))         # Arredondamento para cima -> 5

# Geração de dados aleatórios
numero_aleatorio = random.randint(1, 100) # Gera inteiro entre 1 e 100
```

Técnicas Fundamentais de Depuração (Debugging)

1. **Print Debugging:** Técnica rudimentar mas comum, que consiste em incluir impressões temporárias de variáveis no meio do script para verificar o estado dos dados.
2. **Nativo breakpoint():** Inserido no código (a partir do Python 3.7), ele interrompe imediatamente a execução no local exato e inicia o console de depuração interativo PDB (Python Debugger), permitindo avaliar variáveis linha a linha.

10. ENTRADA/SAÍDA DE ARQUIVOS E TRATAMENTO DE EXCEÇÕES

O tratamento de erros (exceções) previne o encerramento inesperado do software causado por cenários imprevistos ou falhas de infraestrutura.

Estrutura Robusta: try / except / else / finally

```
try:
    denominador = int(input("Digite o divisor: "))
    calculo = 100 / denominador
except ZeroDivisionError:
    print("Erro crítico: Divisões por zero não são matematicamente aceitas.")
except ValueError:
    print("Erro crítico: Entrada inválida. Digite apenas números inteiros.")
else:
    # Executado exclusivamente se o bloco 'try' NÃO disparou nenhum erro
    print(f"Cálculo executado com total sucesso. Resultado: {calculo}")
finally:
    # Executado SEMPRE, independentemente do sucesso ou ocorrência de erros
    print("Finalizando fluxo de operações monetárias.")
```

Gerenciamento Seguro de Arquivos com a instrução Context Manager (with open)

A palavra-chave `with` implementa um gerenciador de contexto. Ela garante que os descritores de arquivos abertos no Sistema Operacional sejam fechados automaticamente ao fim do bloco, evitando vazamentos de memória mesmo sob cenários de erro abruptos.

```
# Gravação de arquivos (Modo 'w' - Write: Sobrescreve o arquivo existente)
with open("relatorio_log.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write("LOG #001 - Conexão efetuada com sucesso às 14:00\n")
    arquivo.write("LOG #002 - Transação concluída com persistência de dados\n")

# Leitura de arquivos de forma defensiva (Modo 'r' - Read)
try:
    with open("relatorio_log.txt", "r", encoding="utf-8") as arquivo:
        linhas = arquivo.readlines()
        for linha in linhas:
            print(f"Processado do arquivo: {linha.strip()}")
except FileNotFoundError:
    print("O arquivo de destino não pôde ser localizado no diretório atual.")
```

11. CONTEÚDO AVANÇADO: COMPREENSÃO DE LISTAS (LIST COMPREHENSIONS)

Conceito amplamente exigido em testes teóricos de proficiência. É uma sintaxe curta e idiomática usada para mapear ou filtrar listas existentes gerando novas coleções sem a verbosidade de laços for tradicionais.

```
# Exemplo 1: Gerar lista com quadrado dos números (Sintaxe Tradicional)
quadrados = []
for x in range(1, 6):
    quadrados.append(x**2)

# Equivalente Pythonico via List Comprehension:
quadrados_pythonicos = [x**2 for x in range(1, 6)]
print(quadrados_pythonicos) # [1, 4, 9, 16, 25]

# Exemplo 2: List Comprehension aplicando filtros (Apenas números pares)
pares_filtrados = [n for n in range(11) if n % 2 == 0]
print(pares_filtrados) # [0, 2, 4, 6, 8, 10]
```

12. CHECKLIST DE REVISÃO RÁPIDA PRÉ-PROVA

- ☐ A função `input()` sempre retorna dados textuais (`str`). Operações matemáticas exigem casting (conversão).
- ☐ Estruturas de repetição, condicionais e funções obrigatoriamente terminam com dois pontos (`:`) em sua assinatura de bloco.
- ☐ A indentação (recuo do texto através de 4 espaços) substitui as chaves das linguagens C/Java, ditando o escopo do código.
- ☐ Strings e listas possuem indexação baseada em zero (0). O fatiamento superior é excludente (ex: `[0:3]` lê posições 0, 1 e 2).
- ☐ Tuplas utilizam parênteses e são **imutáveis**. Listas utilizam colchetes e são amplamente **mutáveis**.